SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: PRINCIPLES OF OPERATING SYSTEM

Practical No.5

Aim:
1.Multi-threading and Fibonacci Generation
Code:

```
import threading
print("Deepak Sahani")
def generate_fibonacci(n):
    a, b = 0, 1
    sequence = []

    for _ in range(n):
        sequence.append(a)
        a, b = b, a + b

    return sequence

def task(n):
    result = generate_fibonacci(n)
    print(f"Fibonacci({n}) = {result}")

if __name__ == "__main__":
    print("Multi-threaded Fibonacci Sequence Generator\n")

    values = [4, 5, 6]
    threads = []

    for n in values:
        t = threading.Thread(target=task, args=(n,))
        threads.append(t)
        t.start()

    for t in threads:
        t.join()

    print("\nAll threads completed.")
```

Output:

Name: Deepak Sahani
Roll No. S108

```
C:\Deepak\OS>python P5.py
Deepak Sahani
Multi-threaded Fibonacci Sequence Generator

Fibonacci(4) = [0, 1, 1, 2]
Fibonacci(5) = [0, 1, 1, 2, 3]
Fibonacci(6) = [0, 1, 1, 2, 3, 5]

All threads completed.
```

Aim:
Implement multi-threading to generate and print Fibonacci sequences.
code:

```
import threading

def fibonacci(n):
    a, b = 0, 1
    sequence = []

    for _ in range(n):
        sequence.append(a)
        a, b = b, a + b

    return sequence

def task(n):
    print(f"Fibonacci({n}) = {fibonacci(n)}")

if __name__ == "__main__":
    values = [4, 5, 6]
    threads = []

    for n in values:
        t = threading.Thread(target=task, args=(n,))
        threads.append(t)
        t.start()

    for t in threads:
        t.join()

    print("All threads completed.")
```

Output:

Name: Deepak Sahani
Roll No. S108

```
C:\Deepak\OS>python P5_234.py
Multi-threaded Fibonacci Sequence
-----------------------------------
Fibonacci(4) = [0, 1, 1, 2]
Fibonacci(5) = [0, 1, 1, 2, 3]
Fibonacci(6) = [0, 1, 1, 2, 3, 5]
All Fibonacci threads completed.
```

Aim:
Explore thread safety, synchronization when accessing shared variables.
Code:

```
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter

    for _ in range(100000):
        with lock:
            counter += 1

threads = []

for _ in range(5):
    t = threading.Thread(target=increment)
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print("Final Counter Value:", counter)
```

Output:

```
Thread Safety and Synchronization
-----------------------------------
Final Counter Value: 500000
```

Aim:
Introduce concepts of thread pooling and task delegation.
Code:

```
from concurrent.futures import ThreadPoolExecutor
```

Name: Deepak Sahani
Roll No. S108

```
def task(n):
    return f"Task {n} completed by {threading.current_thread().name}"

import threading

if __name__ == "__main__":
    tasks = [1, 2, 3, 4, 5]

    with ThreadPoolExecutor(max_workers=3) as executor:
        results = executor.map(task, tasks)

        for result in results:
            print(result)

    print("All tasks completed.")
```

Output:

```
Thread Pooling and Task Delegation
----------------------------------
Task 1 completed by ThreadPoolExecutor-0_0
Task 2 completed by ThreadPoolExecutor-0_0
Task 3 completed by ThreadPoolExecutor-0_0
Task 4 completed by ThreadPoolExecutor-0_0
Task 5 completed by ThreadPoolExecutor-0_1
```

Aim:

1. Write a multi-threaded Python program where each thread calculates the factorial of a different number. Display the factorial result from each thread separately.

Code:

```
import threading

lock = threading.Lock()

def factorial(n):
    result = 1

    for i in range(1, n + 1):
        result *= i

    with lock:
        print(f"Factorial of {n} = {result}")

if __name__ == "__main__":
    numbers = [5, 6, 7, 8]
    threads = []
```

Name: Deepak Sahani
Roll No. S108

```
    for n in numbers:
        t = threading.Thread(target=factorial, args=(n,))
        threads.append(t)
        t.start()

    for t in threads:
        t.join()

    print("All factorial threads completed.")
```

Output:

```
C:\Deepak\OS>python P5_45.py
Factorial of 5 = 120
Factorial of 6 = 720
Factorial of 7 = 5040
Factorial of 8 = 40320

Even Numbers:
2
4
6
8
10
```

Aim;

2. Write a program to print even-odd numbers and reverse a string using the concept of Multithreading.

Code:

```
import threading

lock = threading.Lock()

def print_even():
    with lock:
        print("Even Numbers:")
        for i in range(2, 11, 2):
            print(i)

def print_odd():
    with lock:
        print("Odd Numbers:")
        for i in range(1, 11, 2):
            print(i)
```

Name: Deepak Sahani
Roll No. S108

```
def reverse_string(text):
    with lock:
        print("Original String:", text)
        print("Reversed String:", text[::-1])

if __name__ == "__main__":
    t1 = threading.Thread(target=print_even)
    t2 = threading.Thread(target=print_odd)
    t3 = threading.Thread(
        target=reverse_string,
        args=("Python Programming",)
    )

    t1.start()
    t2.start()
    t3.start()

    t1.join()
    t2.join()
    t3.join()

    print("All threads completed.")
```

Output: